학습 노트: Repository 패턴 (DB 접근 래핑 + 의존성 역전)

작성일: 2026-06-06 **관련 파일**: `dev2/repositories/scan_repository.py`, `dev2/repositories/__init__.py`, `dev2/nodes/store.py`, `dev2/nodes/aggregate.py`, `dev2/db.py`, `dev2/tests/test_aggregate.py` **복습 예정**: D+1 (6/7), D+3 (6/9), D+7 (6/13)

한 줄 요약

에이전트(노드) 코드가 **SQL을 직접 만지지 않고**, "이런 일을 해줘"라는 **주문서(인터페이스)** 만 알게 하고, 실제 DB 작업은 뒤에 숨은 **담당 부서(구현체)** 가 처리하게 하는 패턴입니다.

비유 🔌

- **인터페이스(ScanRepository)** = 벽에 있는 **표준 콘센트**. 가전제품(노드)은 콘센트 모양만 알면 됩니다.
- **구현체(SqlAlchemyScanRepository)** = 콘센트 뒤의 **실제 발전소**. 화력이든 태양광이든(PostgreSQL이든 Mock이든) 콘센트 모양만 같으면 가전제품은 바뀔 필요가 없습니다.
- **팩토리(get_scan_repository)** = "지금은 어느 발전소에 연결할지" 정해주는 **배전반**.

선풍기(노드)는 "발전소가 화력인지 태양광인지" 몰라도 됩니다. 콘센트(인터페이스)에만 의존하니까요. 이게 **의존성 역전(DIP)** 입니다.

명확한 개념

Repository 패턴: 데이터 저장/조회 로직을 한 곳(Repository)으로 모아, 비즈니스 코드가 **추상 인터페이스**를 통해서만 DB에 접근하게 하는 설계. SQL/ORM 세부사항이 비즈니스 코드에 새어나오지 않습니다.

의존성 역전 원칙(DIP): 상위 모듈(노드)이 하위 모듈(DB 코드)에 직접 의존하지 않고, **둘 다 추상(인터페이스)에 의존**하게 만든다. CLAUDE.md §2의 "의존성은 항상 안쪽(도메인)을 향한다"가 이것.

```
[Use Case]  nodes/store.py, nodes/aggregate.py
      │ 의존 (추상만)
      ▼
[Interface] ScanRepository (ABC)       ← 노드는 여기까지만 안다
      ▲ 구현
      │
[Adapter]   SqlAlchemyScanRepository → dev2.db (SQL + asyncpg)
```

왜 쓰나요?

없으면 노드 코드가 `from dev2.db import insert_scan_execution ...` 처럼 SQL 함수를 직접 붙들고 삽니다. 그러면:

- DB를 바꾸거나(예: 다른 드라이버) Mock으로 갈아끼우려면 **노드 코드를 다 고쳐야** 함
- 테스트할 때마다 SQL 함수를 일일이 가짜로 바꿔야 함 (실제로 `test_aggregate`가 함수 2개 × 7곳 = 14번 패치하고 있었음)
- "비즈니스 로직"과 "DB 접근"이 뒤섞여 책임이 흐려짐

있으면 노드는 콘센트(인터페이스)만 보고, 발전소(구현체)는 얼마든지 교체 가능 → 변경에 강하고 테스트가 쉬워짐.

언제 쓰나요?

- **적합**: DB 접근이 여러 곳에서 일어나고, 테스트에서 DB를 Mock해야 하며, 나중에 저장소가 바뀔 수 있을 때.
- **부적합**: 진짜 작은 스크립트, DB 호출이 한두 군데뿐이고 교체/테스트 수요가 없을 때 (괜한 레이어가 오히려 복잡도만 늘림 — over-engineering).

어떻게 동작하나요? (코드 흐름)

1단계 — 인터페이스 정의 (`dev2/repositories/scan_repository.py`)

```
class ScanRepository(ABC):           # ABC = 추상 베이스 클래스 (직접 인스턴스화 불가)
    @abstractmethod
    async def insert_execution(self, task_id, pr_id, execution_ms, ...) -> int:
...
    @abstractmethod
    async def fetch_results_by_task(self, task_id) -> list[dict]: ...
    # ... insert_issues / insert_summary / insert_aggregated_result
```

`@abstractmethod`는 "이 메서드는 자식이 반드시 구현해야 한다"는 강제 규칙입니다.

2단계 — 구현체 (같은 파일)

```
class SqlAlchemyScanRepository(ScanRepository):
    async def fetch_results_by_task(self, task_id):
        from dev2.db import fetch_scan_results_by_task  # lazy import
        return await fetch_scan_results_by_task(task_id)
```

실제 SQL은 여전히 `dev2.db`에 있고, 구현체는 거기에 **위임(delegate)** 만 합니다(이번 단계 목표는 "seam 도입"). `from ... import`를 메서드 **안에서** 하는 lazy import는, asyncpg가 안 깔린 테스트 환경에서도 모듈을 불러올 수 있게 해줍니다.

3단계 — 팩토리

```
_default_repository = SqlAlchemyScanRepository()
def get_scan_repository() -> ScanRepository:
    return _default_repository
```

4단계 — 노드가 인터페이스만 사용 (`dev2/nodes/aggregate.py`)

```
from dev2.repositories import get_scan_repository   # SQL 함수 직접 import 사라짐

async def _load_results(state):
```

```
    repo = get_scan_repository()
    db_results = await repo.fetch_results_by_task(task_id)   # 콘센트만 봄
```

5단계 — 테스트 단순화 (dev2/tests/test_aggregate.py)

```
# 전: 함수 2개를 매번 따로 패치 (insert_aggregated_result +
fetch_scan_results_by_task)
# 후: 콘센트 하나만 가짜로 교체
with patch("dev2.nodes.aggregate.get_scan_repository",
return_value=_mock_repo(...)):
    result = await aggregate_node(state)
```

테스트 seam = "테스트가 가짜를 끼워넣는 단 하나의 지점". 패치 지점이 1개로 줄어 테스트가 단순·견고해집니다.

프로젝트에서

파일	역할
dev2/repositories/scan_repository.py	인터페이스(ScanRepository) + 구현체(SqlAlchemyScanRepository) + 팩토리(get_scan_repository)
dev2/nodes/store.py _persist	raw db 함수 대신 repo.insert_execution/insert_issues/insert_summary 사용
dev2/nodes/aggregate.py	repo.fetch_results_by_task(DB 릴레이) + repo.insert_aggregated_result 사용
dev2/db.py	실제 SQL이 사는 곳(구현체가 위임). import 시 DATABASE_URL을 stdout에 찍던 print 제거(자격증명 노출 방지)

검증: dev2 테스트 184 passed (실패 14건은 Airflow 미설치 건으로 무관). get_scan_repository()가 ScanRepository의 인스턴스를 반환하는 것까지 확인.

연관 개념

- **ABC / abstractmethod** → "이 모양을 반드시 따르라"고 강제하는 추상 클래스. 콘센트 규격서.
- **의존성 역전(DIP)** → 상위·하위가 모두 추상에 의존. Clean Architecture의 핵심(CLAUDE.md §2).
- **lazy import** → 무거운/선택적 의존성(asyncpg)을 실제 쓸 때만 불러오기. [[store_node_patterns]] 참고.
- **테스트 seam** → 가짜를 끼우는 단일 지점. 패치 수를 줄여 테스트를 단순화.
- **DB 릴레이** → 에이전트 간 결과를 DB로 전달하는 패턴. 이번에 그 조회도 Repository를 거치게 됨. [[db-relay-agent-handoff]] 참고.

복습 퀴즈 (3문항)

1. **(이해)** 노드가 from dev2.db import insert_scan_execution처럼 SQL 함수를 직접 import하면 무엇이 불편해지나요? Repository를 쓰면 그 불편이 어떻게 해결되나요?

2. **(적용)** 만약 PostgreSQL 대신 테스트에서 메모리 기반 가짜 저장소를 쓰고 싶다면, 어떤 클래스를 새로 만들고 어디를 바꾸면 될까요? (힌트: `ScanRepository`, `get_scan_repository`)

3. **(비교)** 리팩터링 전 `test_aggregate`는 한 테스트에서 함수 2개를 따로 패치했는데, 후에는 1개만 패치합니다. 이게 가능해진 이유는 무엇인가요? ('테스트 seam' 단어를 써서 설명해 보세요.)